Ashoka University | Spring 2026

1 May 2026

RESERVOIR COMPUTING WITH ITERATIVE POLYNOMIALS

SAHAANA VIJAY

Advisors: Dr. Bikram Phookun, Dr. Abhinav Gupta, Dr. Ramakrishna Ramaswamy

RESERVOIR COMPUTING WITH ITERATIVE POLYNOMIALS

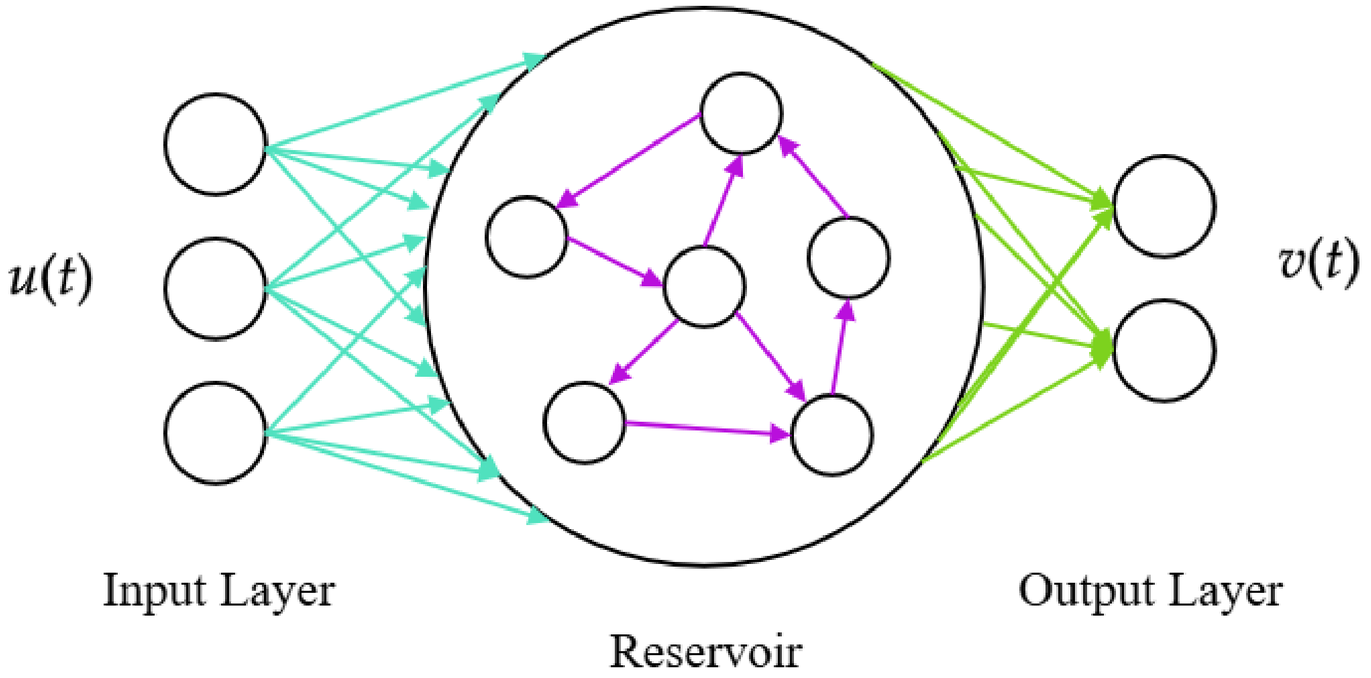


Fig 1. A general reservoir computer schematic.

WHY RESERVOIR COMPUTING?

- ML is a powerful tool yet predicting the physical world remains an open challenge

- Dynamical systems like weather, turbulent flows, and gravitational N-body systems are chaotic being sensitive to initial conditions and making long-term prediction fundamentally hard

- Two natural approaches, both hit a wall:

1. **Physics-based:** numerical integration requires knowing the governing equations, but we cannot solve Navier-Stokes for turbulence or the N-body problem beyond a few bodies
2. **Data-based (for e.g., RNNs):** Backpropagation through time is computationally expensive and regular neural networks often need many trainable parameters

- In practice, we sometimes only observe a few variables of the system

HOW DOES A RESERVOIR COMPUTER WORK?

$[u_0, u_1, u_2, .., u_N]$
$[v_0, v_1, v_2, ..., v_N]$

$$r_{enc} = r_{min} + (r_{max} - r_{min}) \cdot \frac{u_i - u_{min}}{u_{max} - u_{min}}$$

Input Encoding

$[r_0, r_1, r_2, ..., r_N]$

Logistic Map Reservoir

$$F(r_{enc}, x_i) = rx(1 - x_i)$$

Reservoir Matrix

$$\Phi$$

$$\begin{bmatrix} [x_0^{r_1}, x_1^{r_1}, x_2^{r_!}, ..., x_M^{r_1}] \\ [x_0^{r_2}, x_1^{r_2}, x_2^{r_2}, ..., x_M^{r_2}] \\ \vdots \\ [x_0^{r_n}, x_1^{r_n}, x_2^{r_n}, ..., x_M^{r_n}] \end{bmatrix}$$

TRAINING A RESERVOIR COMPUTER

The biggest advantage of reservoir computing lies in the fact that we merely train a readout layer through linear regression. We want to learn the transformation

$$\tilde{v} = \mathbf{w} \cdot \Phi$$

Minimising the regularised loss function,

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^{N} \left(\sum_{i=0}^{M} w_i \phi_i - v_i \right)^2 + \lambda \sum_{j=0}^{M} w_j^2$$

We obtain the solution for optimal weights

$$w = (\Phi\Phi^\top + \lambda I)^{-1} \Phi v$$

TRAINING A RESERVOIR COMPUTER

The biggest advantage of reservoir computing lies in the fact that we merely train a readout layer through linear regression. We want to learn the transformation

$$\tilde{v} = \mathbf{w} \cdot \Phi$$

Minimising the regularised loss function,

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^{N} \left(\sum_{i=0}^{M} w_i \phi_i - v_i \right)^2 + \lambda \sum_{j=0}^{M} w_j^2$$

We obtain the solution for optimal weights

$$w = (\Phi\Phi^\top + \lambda I)^{-1}\Phi v$$

- The λI term guarantees invertibility and prevents overfitting
- As $\lambda \to 0$, we get the pseudoinverse solution $\Rightarrow w = \Phi^{+}v$
- λ is kept small but nonzero, this stabilises the inversion without significantly biasing the solution

TEMPORAL TASKS AND FADING MEMORY

For non-temporal tasks, we merely reset the reservoir after each input and do nothing else, so there is no memory of past inputs. We reset the input for temporal tasks too but as we need memory here, we stack past reservoir states with decaying weights, for e.g., exponentially decaying

$$\widetilde{\Phi}_i = [w_0\Phi_{i-m+1},\ w_1\Phi_{i-m+2},\ \ldots,\ w_{m-2}\Phi_{i-2},\ w_{m-1}\Phi_{i-1},\ w_m\Phi_i]$$

$$\text{For e. g., with } w_j = \alpha^j \text{ where } \alpha \in (0,\ 1],$$

$$\widetilde{\Phi}_i = \left[\alpha^{m-1}\phi_{i-m+1},\ \alpha^{m-2}\phi_{i-m+2},\ \ldots,\ \alpha^2\phi_{i-2},\ \alpha\phi_{i-1},\ \phi_i\right]$$

m: memory length (how many past time steps to include)

LOGISTIC MAP AS RESERVOIR

Static Task

Approximating a seventh-degree polynomial

$$f(x) = (x - 3)(x - 3)(x - 1)x(x + 1)(x + 2)(x + 3)$$

over the range $x \in [-3, 3]$ and evaluating its roots. The reservoir is reset to $[x, \dot{x}] = [0, 0]$ after each input to remove memory effects.

Temporal Task

Reconstructing the Lorenz attractor: using one of the variables of the Lorenz system as input to infer the dynamics of another.

Here, we use linearly decaying weights (as applied in the Arun et al. 2024) up to a memory length m.

LOGISTIC MAP RESULTS: STATIC TASK

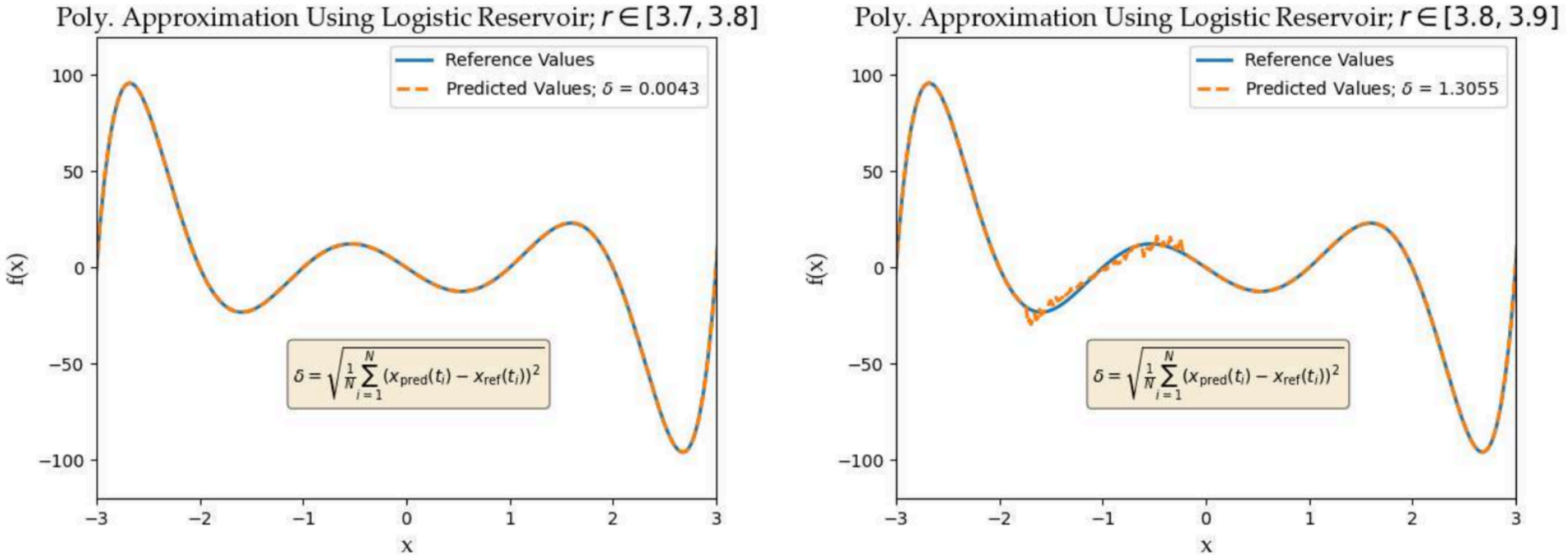
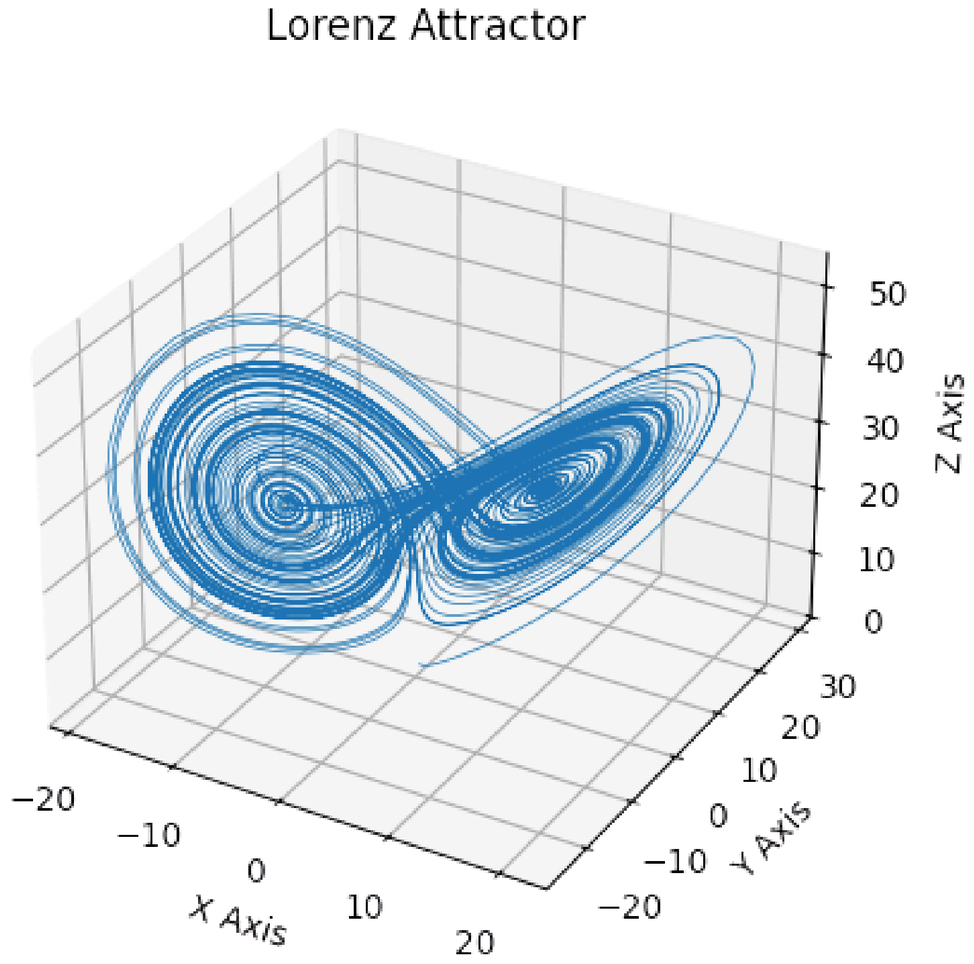


Fig 2: 7th-degree polynomial approximation using logistic map reservoir (a) r ∈ [3.7,3.8], roots exact; (b) r ∈ [3.8,3.9] (Arun et al. 2024), roots approximate.

THE LORENZ SYSTEM



$$\dot{x} = \sigma(y - x)$$

$$\dot{y} = x(\rho - z) - y$$

$$\dot{z} = xy - \beta z$$

LOGISTIC MAP RESULTS: TEMPORAL TASK

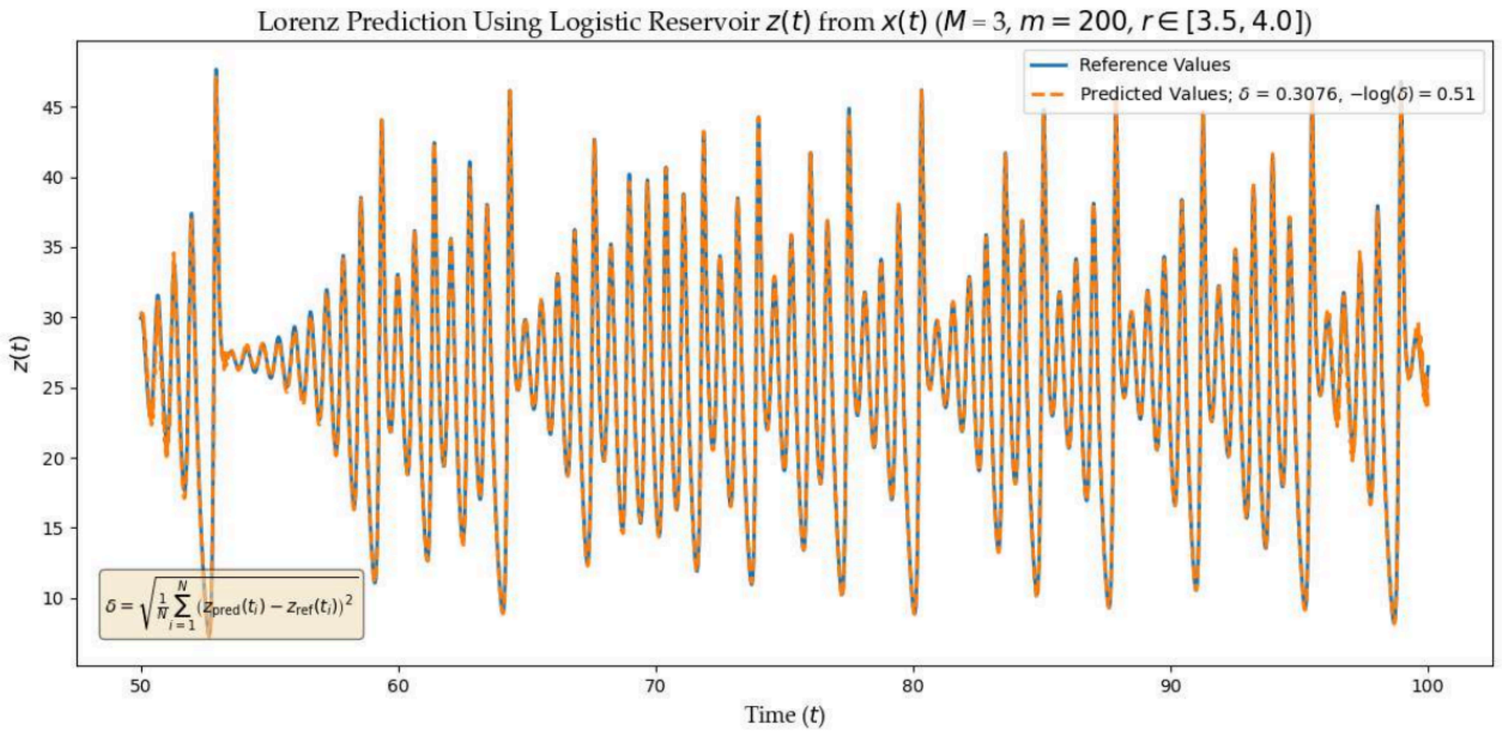


Fig 3: *z(t)* from *x(t)* using logistic map reservoir, r ∈ [3.5, 4.0], *m* = 100; δ ≈ 0.30, best among parameter ranges tested.

We also tested the ranges [3.6, 3.7] and [3.8, 3.9] as in Arun et. al (2024) with δ = 0.79 and 0.85 respectively

LOGISTIC MAP RESULTS: TEMPORAL TASK

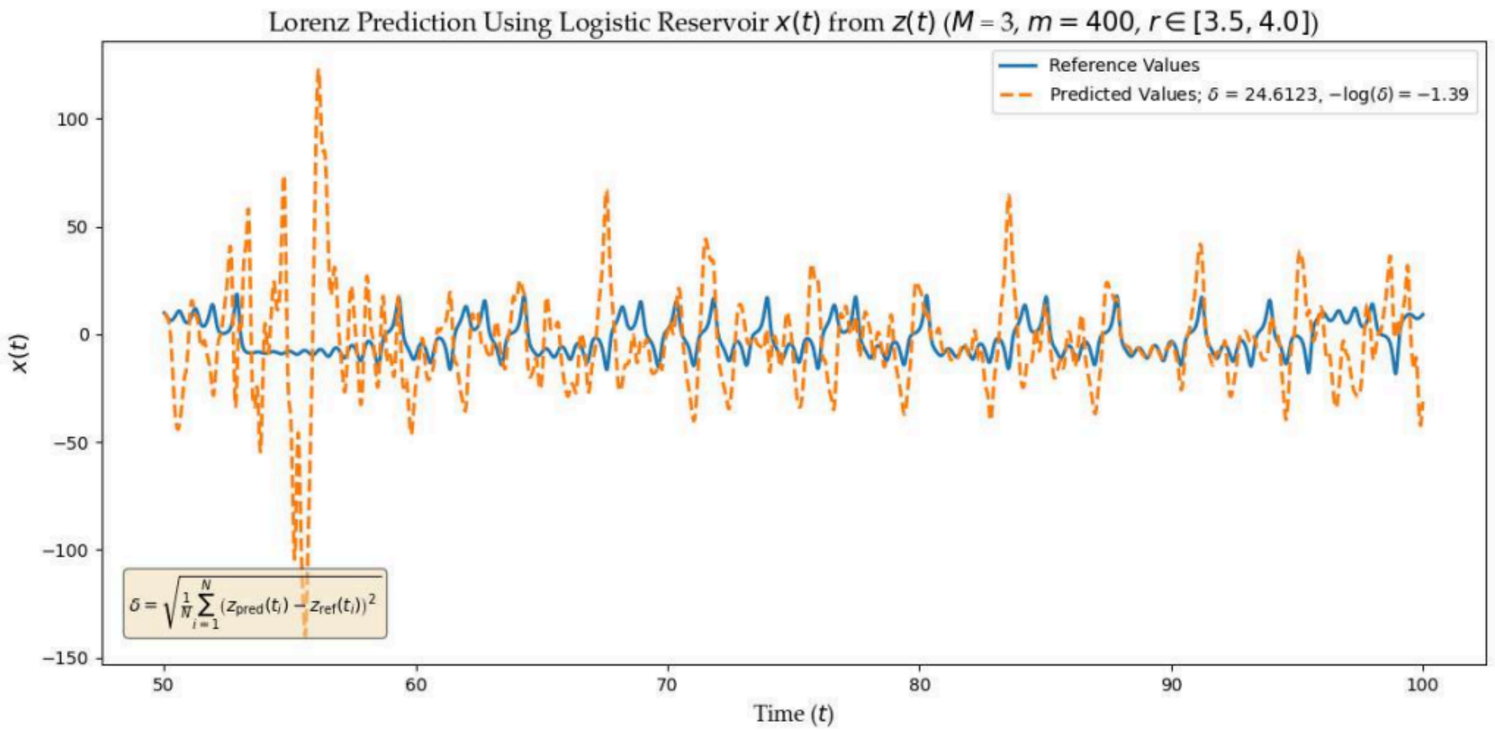


Fig 4: *x(t)* from *z(t)* using logistic map reservoir, r ∈ [3.5, 4.0], *m* = 400; δ ≈ 24.61

The predictions were consistently bad for other parameter ranges as well

LOGISTIC MAP RESULTS: TEMPORAL TASK

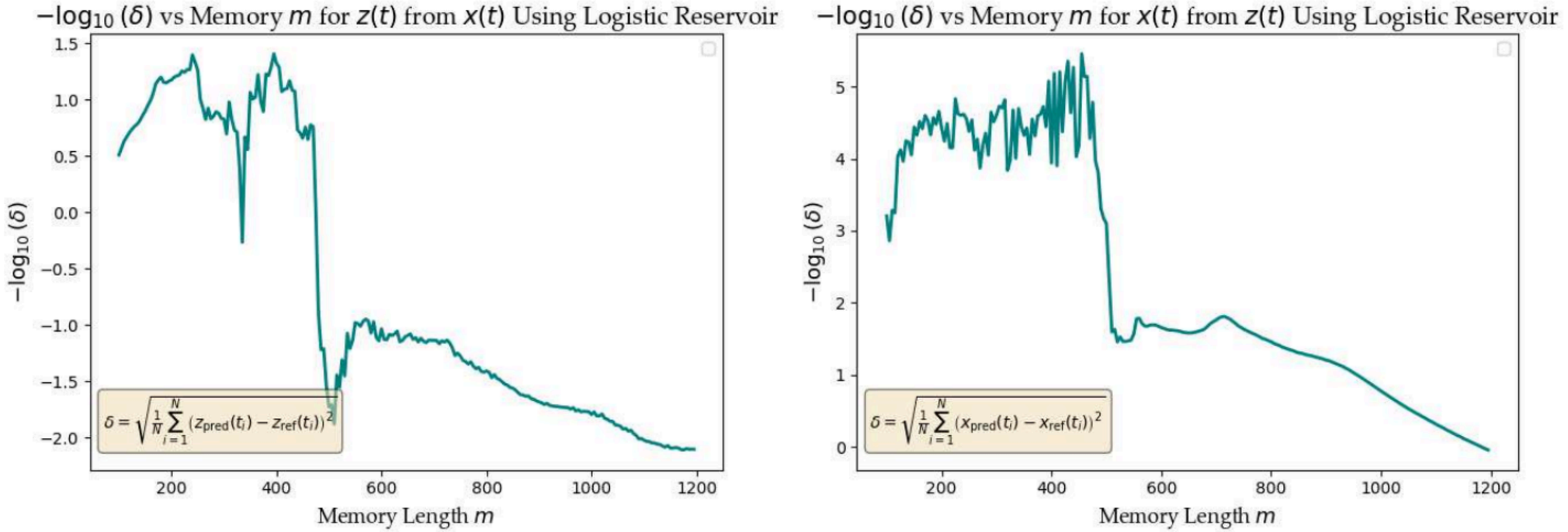


Fig 5: -log10(δ) vs *m* for (a) *z(t)* from *x(t)*: peaks at *m* = 400; (b) *x(t)* from *z(t)*: inconclusive due to task difficulty.

RESULTS FROM ARUN ET AL. & PROBLEMS WITH LOGISTIC MAP

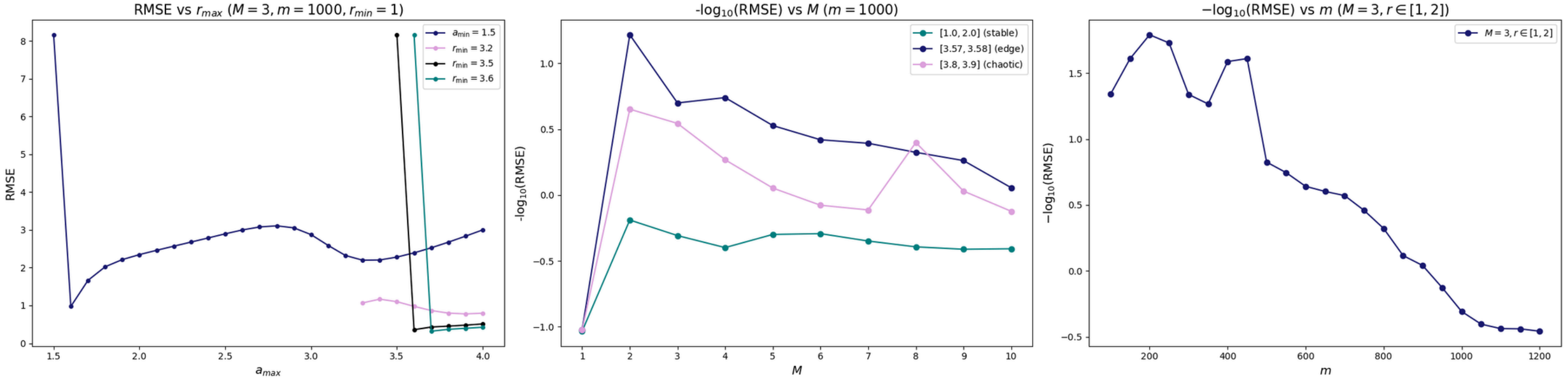


Fig 6: (a) RMSE vs r_{max} ($M = 3$, $m = 1000$); (b) $-\log10(\delta)$ vs M for three parameter ranges; (c) $-\log10(\delta)$ vs m ($r \in [1,2]$) when predicting *y(t)* from *x(t)*. Reproduced from Arun et al. (2024); complete results in the appendix

POLYNOMIAL INTERPRETATION

The iterates of the logistic map are polynomials in r

$$x_1 = 0.25r, x_2 = 0.25r^2 - 0.0625r^3, \ldots$$

The reservoir state Φ built from successive iterates of the map can also be interpreted as linear combinations of polynomial basis functions evaluated across all inputs

$$\Phi_{ji} = \varphi_j(u_i)$$

Now,

- The monomial basis {1, r, r^2} is notorious for being numerically unstable for large M
- The rank of the feature set in the logistic reservoir is unclear, no mathematical guarantee of linear independence

Can we replace monomials with a better conditioned polynomial basis?

CHEBYSHEV POLYNOMIALS

CHEBYSHEV POLYNOMIALS

Definition on $x \in [-1, 1]$:

$$T_0(x) = 1$$
$$T_1(x) = x$$
$$T_2(x) = 2x^2 - 1$$
$$T_3(x) = 4x^3 - 3x$$
$$T_4(x) = 8x^4 - 8x^2 + 1$$

Recurrence relation: $T_{n+1}(x) = 2x \cdot T_n(x) - T_{n-1}(x)$

Key Properties:

- Orthogonal on [-1, 1]

$$\int_{-1}^{1} \frac{T_m(x)\,T_n(x)}{\sqrt{1-x^2}}\,dx = \begin{cases} 0 & m \neq n \\ \pi & m = n = 0 \\ \pi/2 & m = n \neq 0 \end{cases}$$

- Bounded: $|T_n(x)| \leq 1$
- Numerically stable
- Best polynomial approximation (minimax property)
- Can be generated by rotation dynamics!

GENERATING CHEBYSHEV FEATURES USING ROTATIONS

Consider rotation by angle θ in the plane given by $R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

Given an input $x \in [-1, 1]$, we encode it in the rotation matrix by setting

$$\theta = \arccos(x), \qquad \text{so that} \qquad \cos\theta = x, \quad \sin\theta = \sqrt{1 - x^2}$$

Starting from the $(1, 0)$, repeatedly applying of $R(\theta)$ gives us

$$R(\theta)^n \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \cos(n\theta) \\ \sin(n\theta) \end{pmatrix} = \begin{pmatrix} T_n(x) \\ \sqrt{1 - x^2} \cdot U_{n-1}(x) \end{pmatrix}$$

The first component gives the n^{th} Chebyshev polynomial evaluated at the input and corresponds to the n^{th} iterate. The second component is a function of the Chebyshev polynomials of the second kind

GENERATING CHEBYSHEV FEATURES USING RECURRENCE

There is another approach based on the recurrence relation itself. We define the state vector

$$\mathbf{z}_n = \begin{pmatrix} T_n(x) \\ T_{n-1}(x) \end{pmatrix}$$

and the matrix $A(x) = \begin{pmatrix} 2x & -1 \\ 1 & 0 \end{pmatrix}$

The recurrence relation is then equivalent to $z_{n+1} = A(x)z_n$:

$$\mathbf{z}_{n+1} = \begin{pmatrix} 2x & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} T_n(x) \\ T_{n-1}(x) \end{pmatrix} = \begin{pmatrix} 2x\,T_n(x) - T_{n-1}(x) \\ T_n(x) \end{pmatrix} = \begin{pmatrix} T_{n+1}(x) \\ T_n(x) \end{pmatrix}$$

We start with $(T_1(x), T_0(x))^T$ and read off the first set from every iteration

CHEBYSHEV RESULTS: STATIC TASK

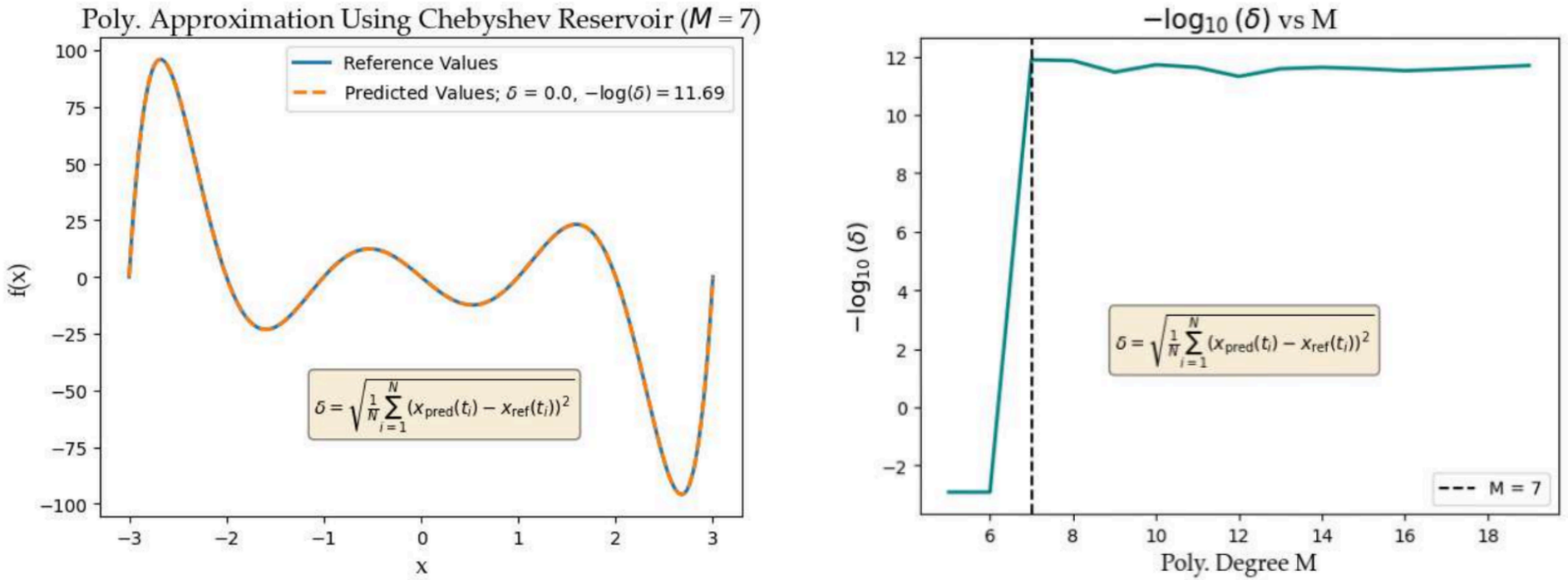
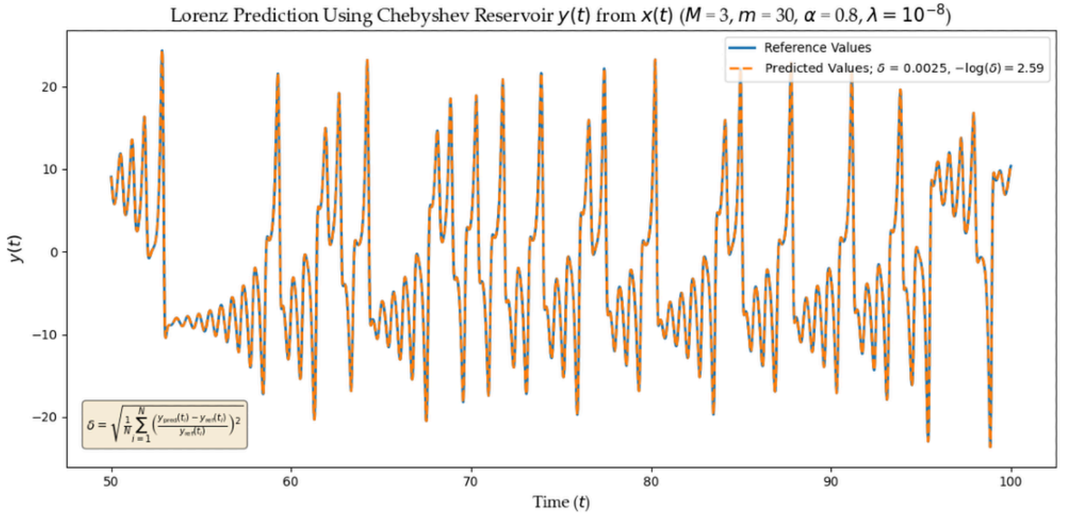


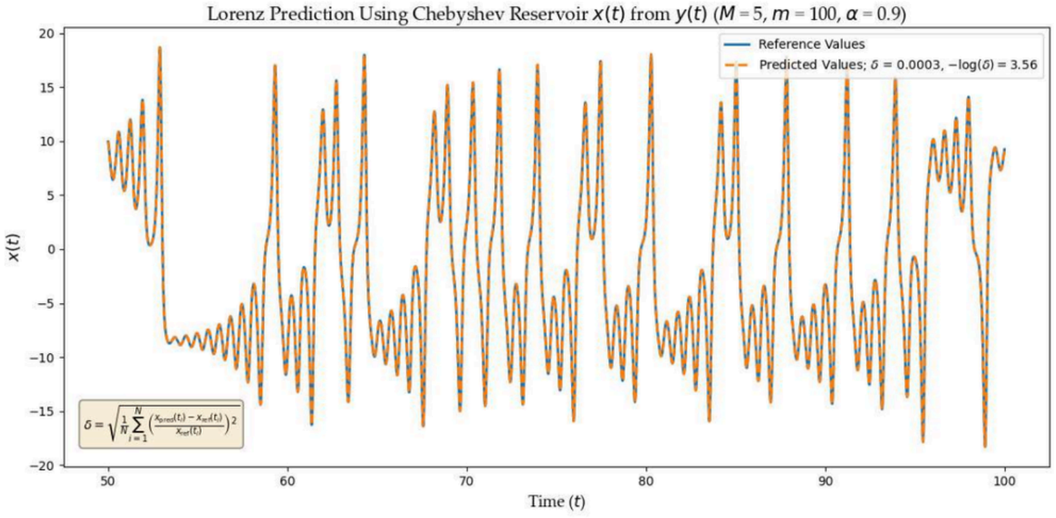
Fig 7: Results for approximating a seventh degree polynomial for x using the Chebyshev reservoir; exact roots. Training time < 1s; this is a trivial task.

CHEBYSHEV RESULTS: TEMPORAL TASK

Using this reservoir, we were able to predict each variable from the other (6 tasks) with smaller errors

Fig. 8: Chebyshev reservoir predictions; (top) *y(t)* from *x(t)*: δ = 0.0025, good agreement with reference; (bottom) *x(t)* from *y(t)*: δ = 0.0003, very close to 0. Both: *m* = 100, α = 0.9, λ = 1e−8.





CHOOSING OPTIMAL PARAMETERS

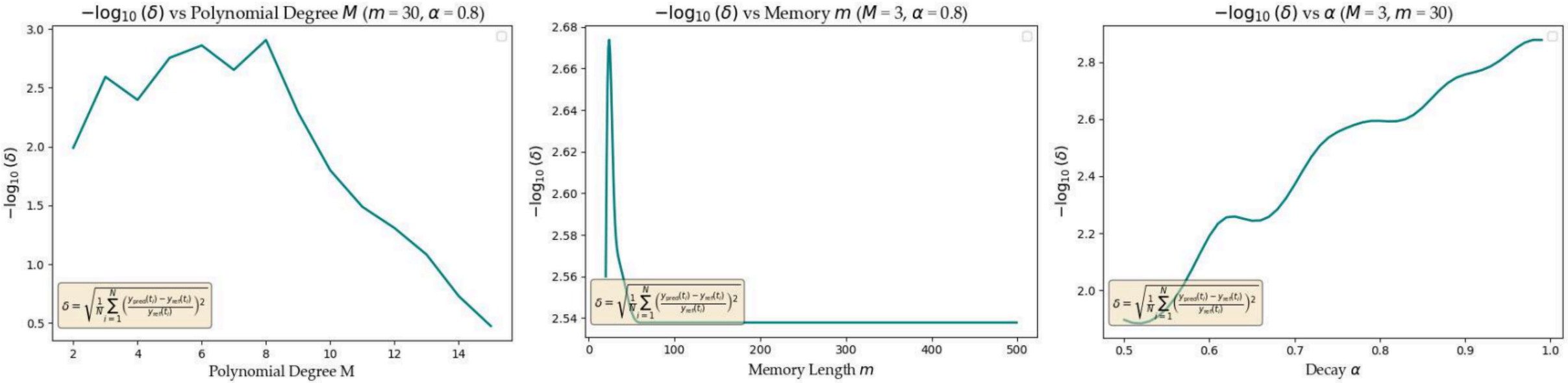


Fig. 9: *y(t)* from *x(t)*: $-\log_{10}(\delta)$ vs (a) *M*: best at M = 8; (b) *m*: best at m = 24; (c) α: predictions improves monotonically with α.

CHOOSING OPTIMAL PARAMETERS

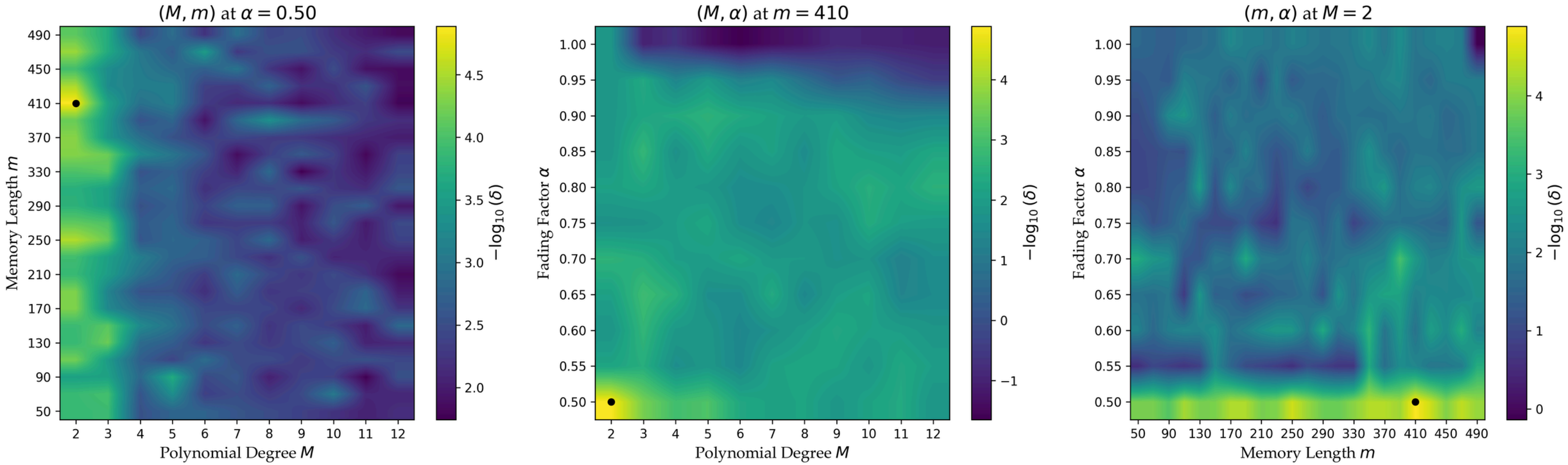


Fig. 10: Complete parameter sweep for *y(t)* from *x(t)*; The optimal parameters were found to be $M = 2$, $m = 410$, $\alpha = 0.5$, $\delta = 0.000013$.

WHERE DOES THIS APPLY?

POSSIBLE FUTURE WORK

1 Using other polynomial reservoirs

Like Hermite or Legendre polynomial basis functions.

2 Inferring $x(t)$ dynamics from $z(t)$

Both reservoirs catastrophically fail in predicting $x(t)$ from $z(t)$. It is very likely that the scale is not the only reason but also the very structure of the Lorenz equations itself. Future work could possibly include defining a new variable such that the zero of $z(t)$ is shifted and inferring dynamics.

REFERENCES

[1] Jaeger — Echo State Networks, GMD Tech. Report 148 (2001)
[2] Pathak et al. — Model-Free Prediction of Large Spatiotemporally Chaotic Systems from Data: A Reservoir Computing Approach., PRL 120, 024102 (2018)
[3] Kong et al. — Machine learning prediction of critical transition and system collapse, PRR 3, 013090 (2021)
[4] Mandal et al. — Machine-Learning Potential of a Single Pendulum, PRE 105, 054203 (2022)
[5] Arun et al. — Reservoir Computing with Logistic Map, arXiv:2401.09501 (2024)
[6] Ratas & Pyragas — Application of next-generation reservoir computing for predicting chaotic systems from partial observations, PRE 109, 064215 (2024)
[7] Abhinav Gupta — Notes on 'Reservoir Computing using Polynomial Features'
[8] Lu et al. — Reservoir observers: Model-free inference of unmeasured variables in chaotic systems, Chaos 27, 041102 (2017)
[9] Chen, Qian and Cui — Time series reconstructing using calibrated reservoir computing, Sci. Rep. 12, 16318 (2022)
[10] Shanaz et al. — Reservoir Computing Using Complex Systems, arXiv:2212.11141 (2022)

Thank You

Acknowledgements:

- Dr. Bikram Phookun, Dr. Abhinav Gupta, Dr. Ramakrishna Ramaswamy
- Physics Department
- Philip Cherian and Souradeep Sengupta
- Dr. Ling-Wei Kong
- My friends and batchmates

$x(t)$ FROM $z(t)$ USING CHEBYSHEV FEATURES

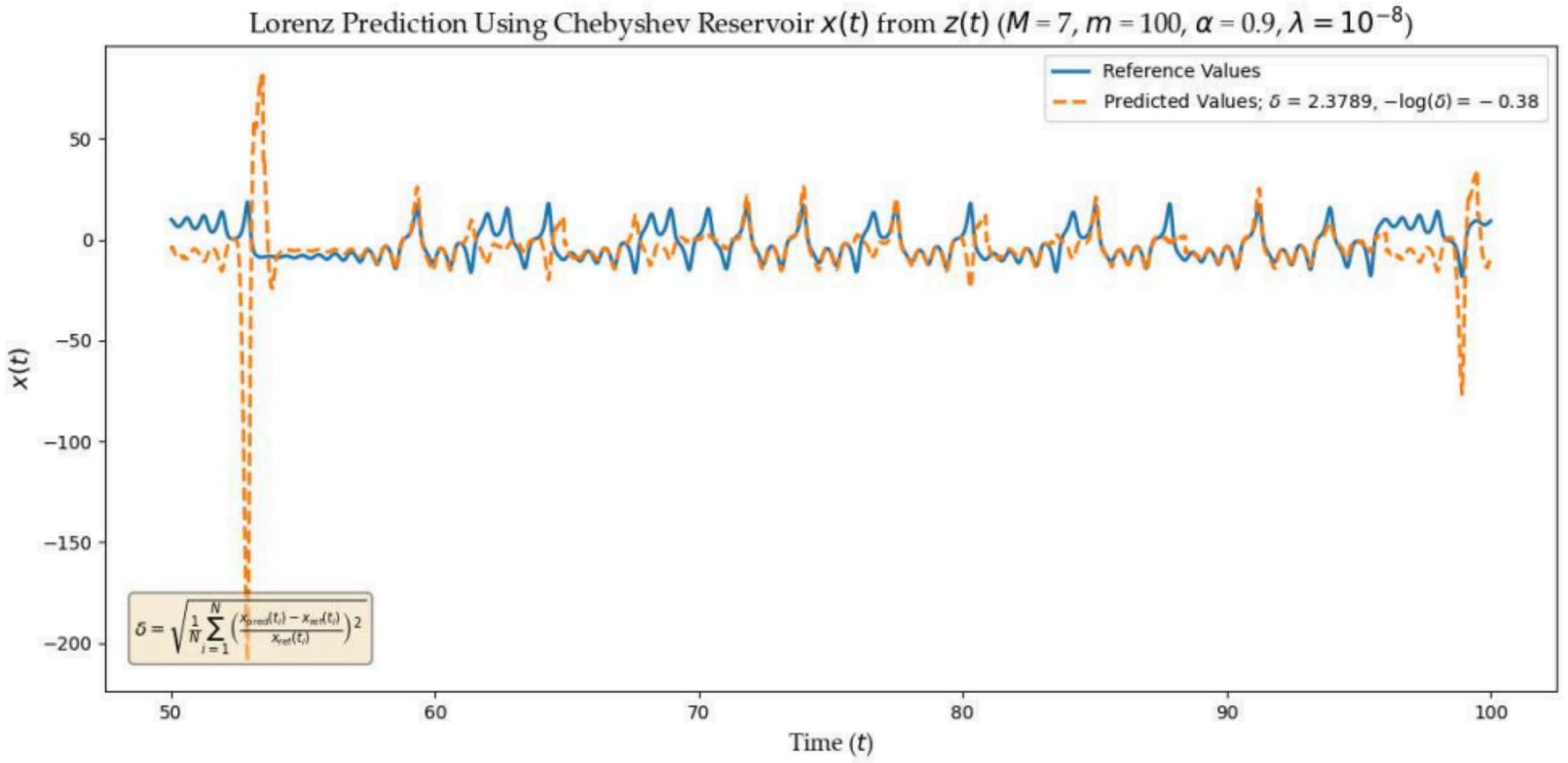


Fig. 12: Predicting $x(t)$ dynamics from $z(t)$ dynamics using Chebyshev features; here, $\delta \approx 2.38$ and $-\log_{10}(\delta) = -0.38$ for $M = 7$, $m = 100$, $\alpha = 0.9$ and $\lambda = 10–8$, which is clearly bad.

x(t) FROM *z(t)* USING CHEBYSHEV FEATURES

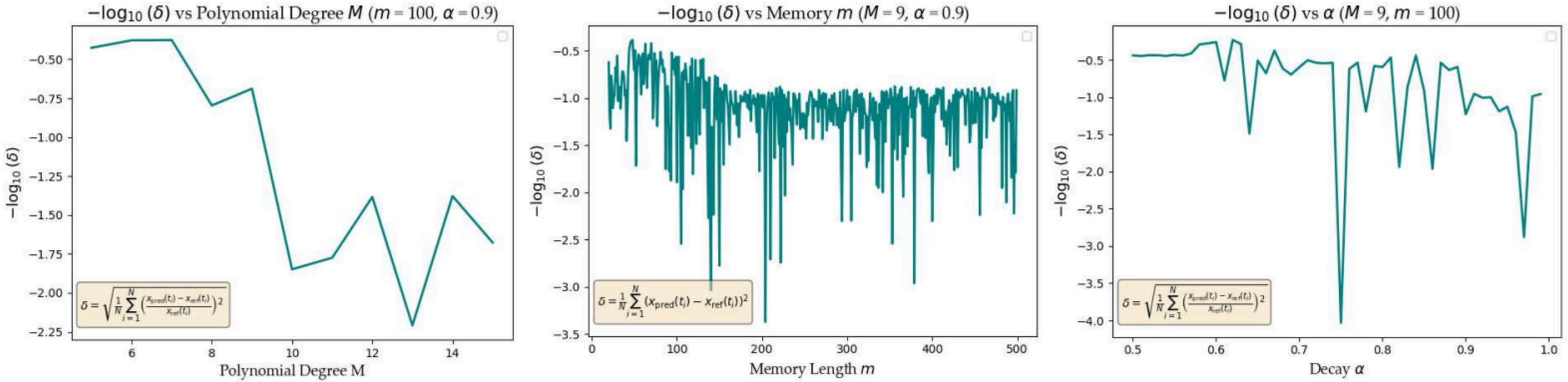


Fig 13: x(t) from z(t): $-\log_{10}(\delta)$ vs (a) *M*: worsens with increasing M; (b) *m*: sensitive but negative throughout; (c) α: negative throughout.

RELATION BETWEEN m and α?

We know $w_i = \alpha^i$

Let us define some threshold ε ≈ 0.01

$$\alpha^{m-1} = \epsilon \implies (m-1)\ln\alpha = \ln\epsilon$$

For ε = 0.01, we have

$$\ln\alpha = \frac{\ln\epsilon}{(m-1)} \implies \alpha = \exp\left(\frac{\ln\epsilon}{(m-1)}\right)$$

$$\alpha = \exp\left(\frac{-4.605}{(m-1)}\right) \approx 1 - \frac{4.6}{m}$$

RELATION BETWEEN m and α?

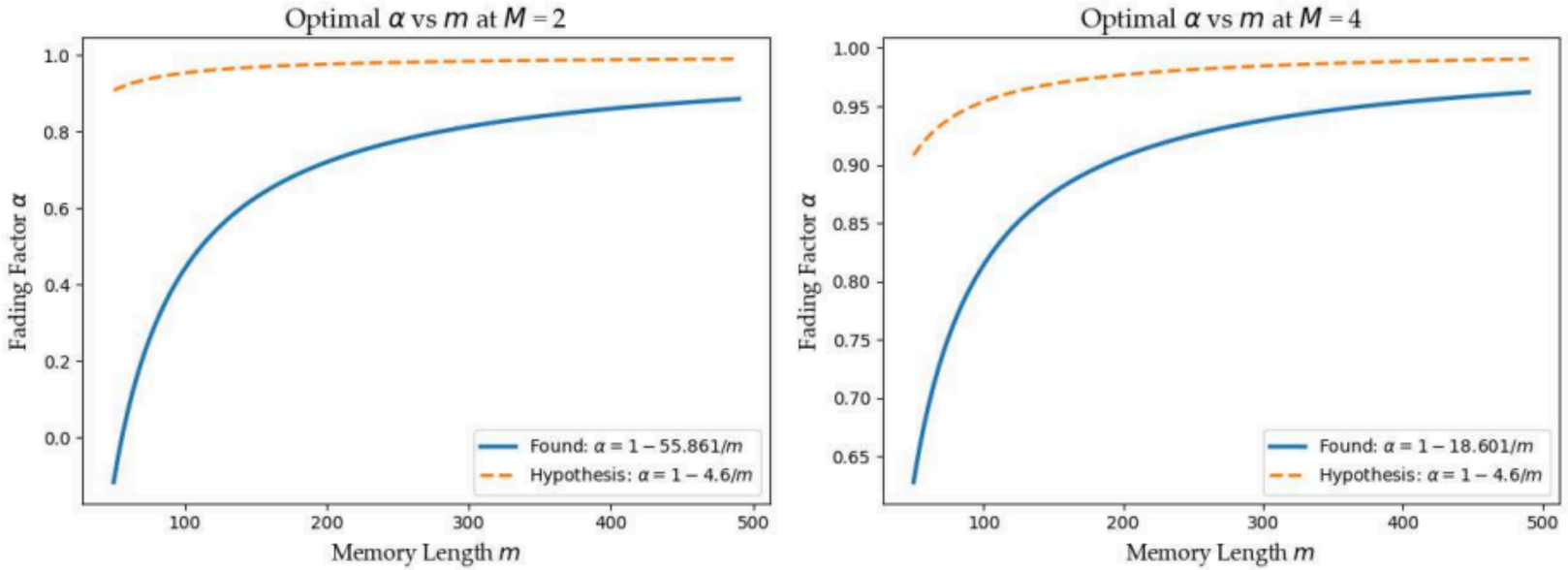


Fig. 11: Optimal α vs m: (left) y(t) from x(t) at M = 2; (right) x(t) from y(t) at M = 4. The thresholds are far lower and the relationship is still not clear.

RELATION BETWEEN m and α?

We can also look at sensitivity around the optimal set of parameters

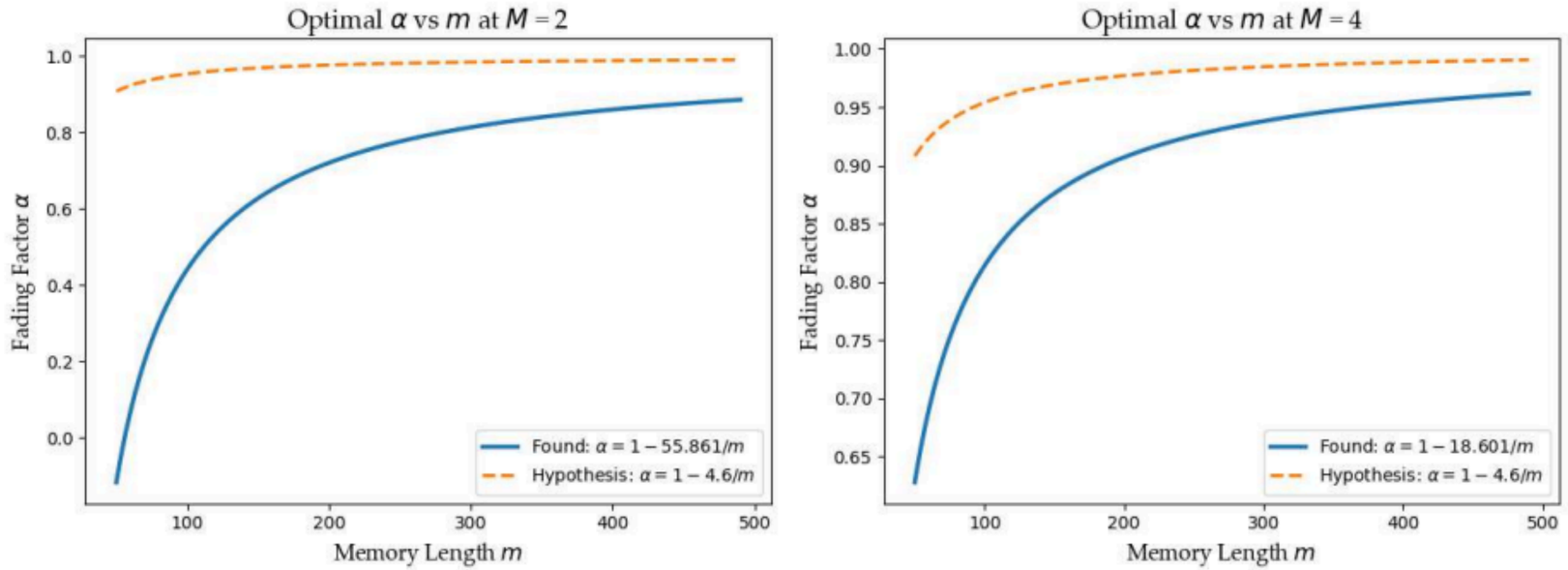


Fig. 11: Optimal α vs m: (left) y(t) from x(t) at M = 2; (right) x(t) from y(t) at M = 4. The thresholds are far lower and the relationship is still not clear.

RESULTS FROM ARUN ET. AL

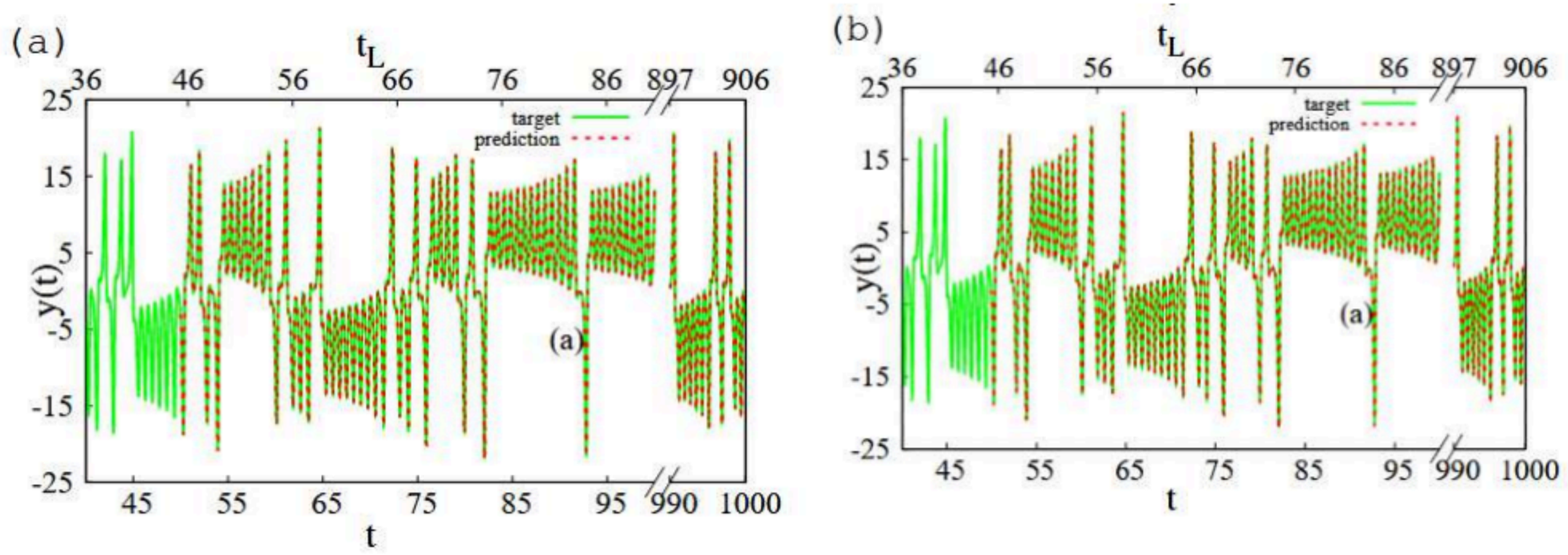


Fig 14. Original plots from the paper 'Reservoir Computing with Logistic Map' by Arun et al. (a) δ (noise) = 0 and (b) δ (noise) = 0.01. Here, $r \in [3.5, 4.0]$, $u \in [-17, 17]$, $M = 3$ and $m = 100$. The Lyapunov time $tL = t \cdot \lambda max$ has also been marked, where $\lambda max = 0.906$ is the maximal Lyapunov exponent of the Lorenz system.

RESULTS FROM ARUN ET. AL

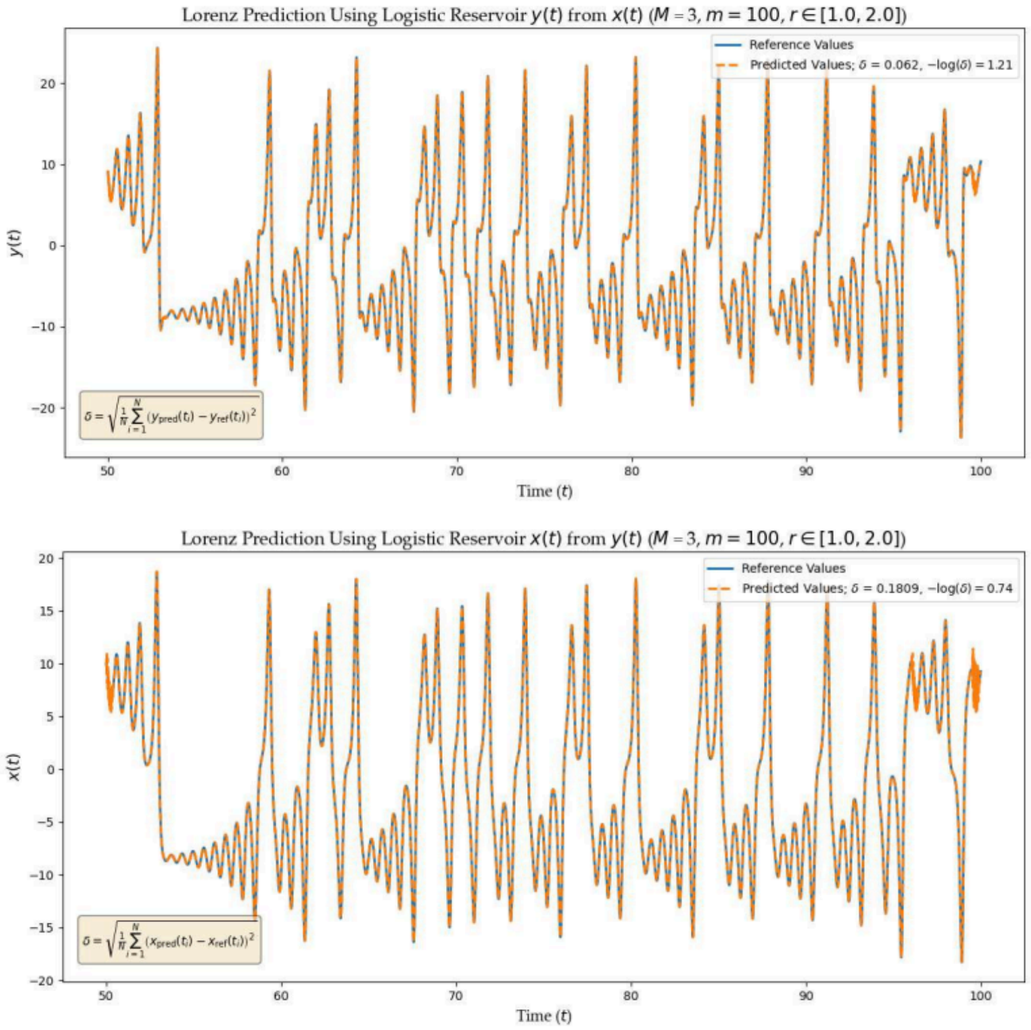


Fig 15. (Top) Predicting *y(t)* dynamics of the Lorenz system with *x(t)* dynamics as input for $m = 100$ and parameter range $r \in [1, 2]$ with $\delta = 0.062$ which is larger as compared to 1.541608e−3 obtained in the paper.

However, we see that the reservoir gives good enough predictions for parameters in the stable regime as well

(Bottom) Predicting *x(t)* dynamics of the Lorenz system with *y(t)* dynamics as input for $m = 100$ and parameter range $r \in [1, 2]$ with $\delta = 0.1809$.

RESULTS FROM ARUN ET. AL

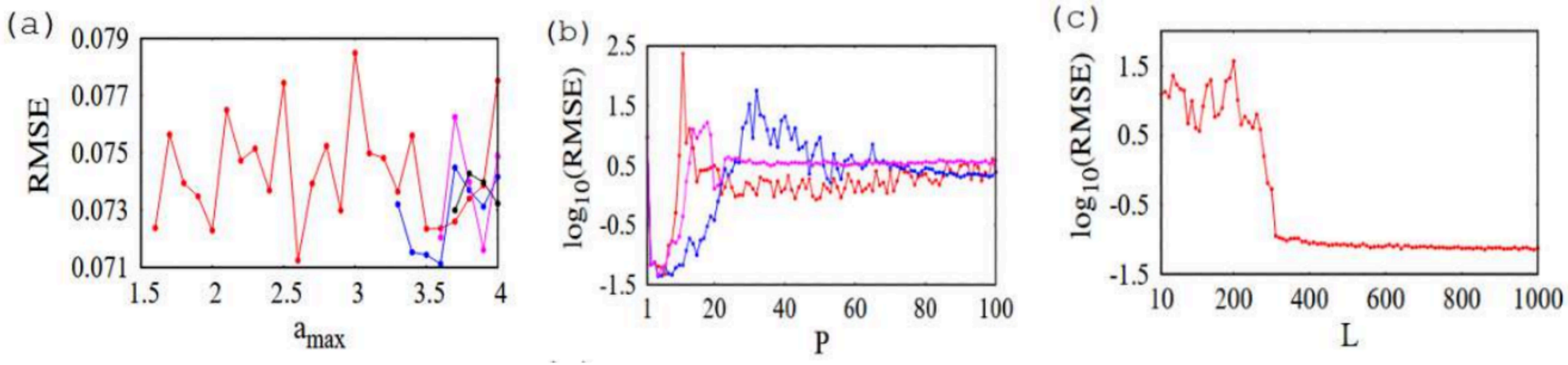


Fig 16. Original plots (a) The root-mean-squared error (RMSE) against rmax when $M = 3$ (here, P) and $m = 1000$, (b) -log10(RMSE) against M from 1 to 100 for $m = 1000$ when (i) r $\in$ [1, 2], (ii) $r \in$ [3.57, 3.58] and (iii) r $\in$ [3.8, 3.9], and (c) -log10(RMSE) against m (here, L) when $M = 3$ and r $\in$ [1, 2]. Here, $u \in$ [1, 2] and the noise strength is 0.01.

x(t) FROM y*(t)* USING CHEBYSHEV FEATURES

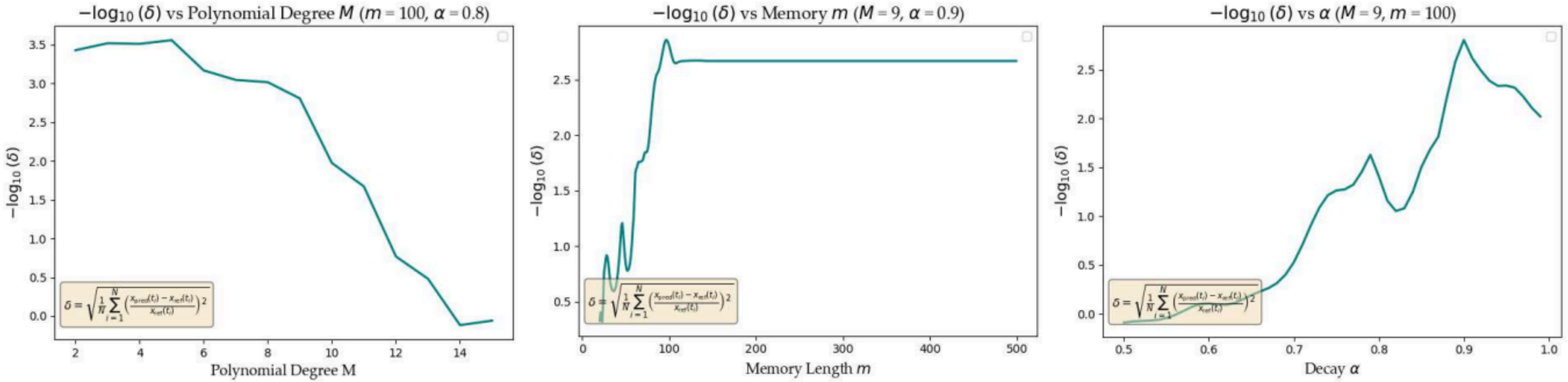


Fig 17. *x(t)* from *y(t)*: $-\log_{10}(\delta)$ vs (a) M: best at $M = 5$; (b) m: best at $m = 100$, saturates beyond; (c) α: best near 0.9.

x(t) FROM y*(t)* COMPLETE PARAMETER SWEEP

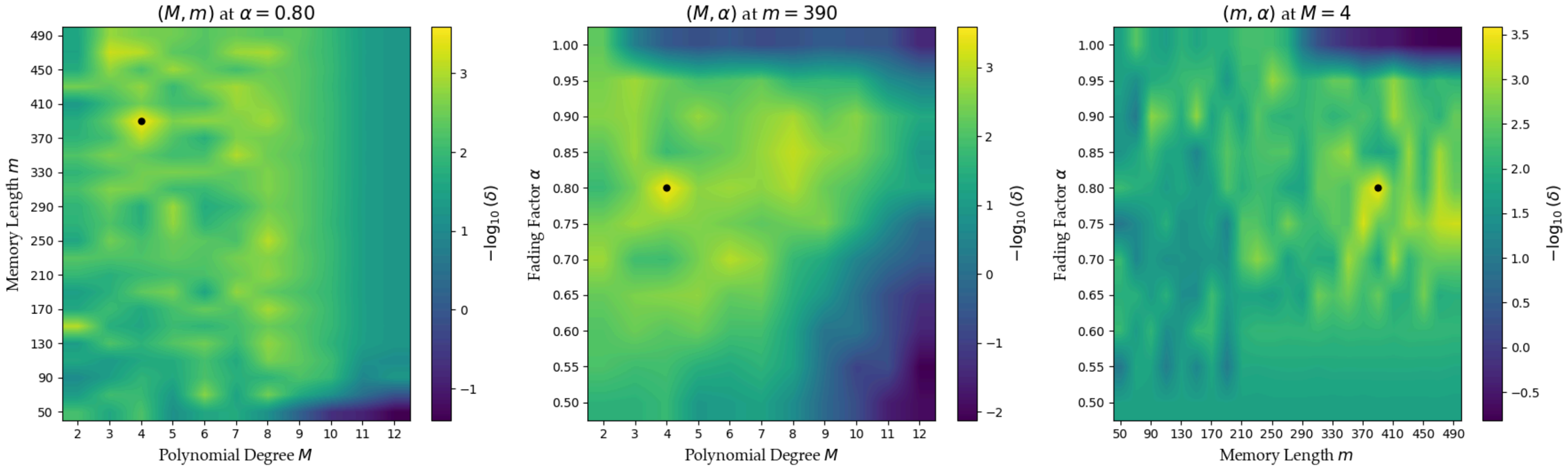


Fig 17. Complete parameter sweep for y(t) from x(t); The optimal parameters were found to be $M = 4$, $m = 390$, $\alpha = 0.800$, $\delta = 0.000260$

FEATURE MAP

$$\Phi = \begin{pmatrix} \phi_0(x_m) & \phi_0(x_{m+1}) & \cdots & \phi_0(x_N) \\ \phi_1(x_m) & \phi_1(x_{m+1}) & \cdots & \phi_1(x_N) \\ \vdots & \vdots & & \vdots \\ \phi_M(x_m) & \phi_M(x_{m+1}) & \cdots & \phi_M(x_N) \\ \hline \alpha\,\phi_0(x_{m-1}) & \alpha\,\phi_0(x_m) & \cdots & \alpha\,\phi_0(x_{N-1}) \\ \alpha\,\phi_1(x_{m-1}) & \alpha\,\phi_1(x_m) & \cdots & \alpha\,\phi_1(x_{N-1}) \\ \vdots & \vdots & & \vdots \\ \alpha\,\phi_M(x_{m-1}) & \alpha\,\phi_M(x_m) & \cdots & \alpha\,\phi_M(x_{N-1}) \\ \hline & \vdots & & \\ \hline \alpha^{m-1}\phi_0(x_1) & \alpha^{m-1}\phi_0(x_2) & \cdots & \alpha^{m-1}\phi_0(x_{N-m+1}) \\ \alpha^{m-1}\phi_1(x_1) & \alpha^{m-1}\phi_1(x_2) & \cdots & \alpha^{m-1}\phi_1(x_{N-m+1}) \\ \vdots & \vdots & & \vdots \\ \alpha^{m-1}\phi_M(x_1) & \alpha^{m-1}\phi_M(x_2) & \cdots & \alpha^{m-1}\phi_M(x_{N-m+1}) \end{pmatrix}$$

Parameters:

N = Total number of data points

M = Maximum polynomial degree

m = Memory length (number of time lags)

α = Fading factor

Feature Matrix Φ:

Dimension: $m(M + 1) \times (N - m + 1)$

- **Rows**: $m(M + 1)$ features
  > $(M + 1)$ Chebyshev polynomials per time lag
  > m time lags in total
- **Columns**: $(N - m + 1)$ valid time points
  > First $(m - 1)$ points lost (need full history window)
  > Valid predictions from time m to time N

Weights:

Dimension: $m(M + 1) \times 1$

One weight per feature in Φ